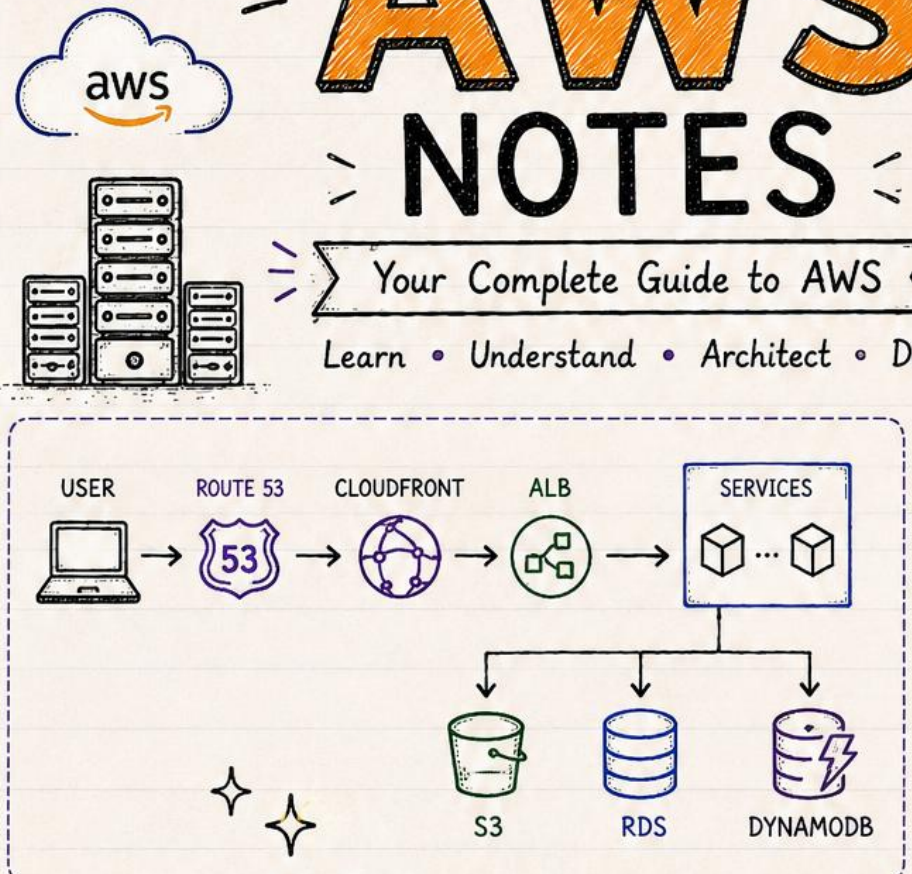
AWS NOTES

Your Complete Guide to AWS

Learn • Understand • Architect • Deploy



AWS IS ABOUT

- Scalability
- Security
- Reliability
- Performance
- Cost Optimization
- Innovation

100 INTERVIEW QUESTIONS

★ BE PREPARED. BE CONFIDENT. GET HIRED. ★

COVERS ALL MAJOR AWS SERVICES

- Compute (EC2, Lambda, ECS, EKS)
- Storage (S3, EBS, EFS, Glacier)
- Database (RDS, DynamoDB, Aurora)
- Networking (VPC, Route 53, CloudFront)
- Security (IAM, KMS, WAF, Shield)
- Monitoring (CloudWatch, CloudTrail)
- Integration (SQS, SNS, EventBridge)
- Management (CloudFormation, Systems Manager)
- DevOps (CodeBuild, CodeDeploy, CodePipeline)
- Analytics (Athena, Redshift, Glue)
- Cost Management (Cost Explorer, Budgets)
- And Many More...

WHAT YOU'LL GET

- ☑ 100 Most Asked Questions
- ☑ Easy to Understand Answers
- ☑ Real-World Use Cases
- ☑ Diagrams & Examples
- ☑ Quick Cheat Sheet
- ☑ Perfect for Interviews

PERFECT FOR

Beginners | Developers | Architects | Interview Preparation | Career Growth

"The Cloud is not just a Technology, it's a Mindset."

Dream. Learn. Build. Repeat.

1. AWS Basics + Cloud Computing

What is AWS?

AWS (Amazon Web Services) is a comprehensive and widely adopted cloud platform provided by Amazon. It offers 200+ fully featured services from data centers globally.

AWS helps individuals, startups, enterprises and governments build scalable, secure and cost-effective applications and infrastructure.

Example

```
# Simple example using S3 (Storage)
aws s3 mb s3://my-first-bucket

# Output
make_bucket: my-first-bucket
```

Did You Know?

AWS was launched in 2006 and is the world's most comprehensive cloud platform!

Key Benefits of AWS

1. **Scalability** : Scale up or down based on demand.
2. **Reliability** : High availability and fault tolerance.
3. **Security** : Built-in security features and compliance.
4. **Cost-Effective** : Pay-as-you-go pricing model.
5. **Global Reach** : Data centers across many regions.
6. **Flexibility** : Wide range of services to build anything.
7. **Innovation** : Regular new features and services.

Where is AWS Used?

- Website Hosting
- Mobile & Web Applications
- Data Storage & Backup
- Big Data Analytics
- Machine Learning
- DevOps & Automation
- IoT, Game Development
- And many more...

Cloud Computing Concepts

1. On-Demand Self-Service
   Users can provision computing resources as needed.
2. Broad Network Access
   Services are available over the network.
3. Resource Pooling
   Provider's resources are shared among users.
4. Rapid Elasticity
   Resources scale quickly in or out.
5. Measured Service
   Usage is measured and billed accordingly.

① IaaS (Infrastructure as a Service)
Provides virtualized computing resources over the internet.
Example: EC2, VPC, S3

② PaaS (Platform as a Service)
Provides a platform to develop, run and manage applications.
Example: Elastic Beanstalk, RDS

③ SaaS (Software as a Service)
Complete applications delivered over the internet.
Example: Gmail, Dropbox, Office 365

AWS Global Infrastructure

→ Regions : A physical location in the world.
→ Availability Zones (AZs) : 2 or more data centers within a region.
→ Edge Locations : Used by CloudFront for content delivery.

Note:
AWS follows a shared responsibility model.
☺

2. IAM + Security

★ What is IAM ?

AWS IAM (Identity and Access Management) allows you to securely control access to AWS services and resources for your users.
It helps you manage permissions and access keys for individuals and applications.

Example

```
# Create an IAM User (via AWS CLI)
aws iam create-user --user-name demo-user

# List Users
aws iam list-users
```

★ Key Components of IAM

1. **Users** : Individual users who access AWS.
2. **Groups** : Collection of users with same permissions.
3. **Roles** : IAM identity that you can assume to get temporary permissions.
4. **Policies** : JSON documents that define permissions.
5. **Identity Providers** : For federated access (AD, Google, SAML, etc.)
6. **MFA (Multi-Factor Authentication)** : Adds extra layer of security.

Did You Know ?

IAM is a global service. Changes made in IAM are reflected across all AWS regions.

☺

IAM Best Practices

- Grant least privilege access.
- Use Groups to manage users.
- Enable MFA for all users.
- Use Roles for applications.
- Regularly review access.
- Avoid using root user.

★ Types of Policies

- **Managed Policies** : AWS or Customer managed.
- **Inline Policies** : Embedded directly into a user, group or role.

★ IAM Policy Structure (JSON)

```
{
  "Version": "2012-10-17",                          → Policy language version
  "Statement": [                                    → One or more statements
    {
      "Effect": "Allow",                            → Allow or Deny
      "Action": ["s3:ListBucket", "s3:GetObject"],  → Actions allowed or denied
      "Resource": ["arn:aws:s3:::my-bucket",        → Resources to which the actions
                   "arn:aws:s3:::my-bucket/*"]        apply
    }
  ]
}
```

★ Security Features in AWS

1. **MFA** : Protects your account by requiring 2nd verification.
2. **Password Policy** : Enforce strong password rules.
3. **Access Keys** : Programmatic access (access key & secret key).
4. **CloudTrail** : Logs all API activity for auditing.
5. **AWS Config** : Monitors and records resource configurations.
6. **GuardDuty** : Threat detection service.

Note :

Never share your AWS credentials.
Use IAM users and Roles instead of root user.

☺

3. EC2 + Auto Scaling

★ What is EC2 ?

Amazon EC2 (Elastic Compute Cloud) provides resizable compute capacity in the cloud.
It allows you to launch virtual servers (instances) and run your applications.

Example

```
# Launch an EC2 Instance (Using AWS CLI)
aws ec2 run-instances \
  --image-id ami-0abcdef1234567890 \
  --count 1  --instance-type t2.micro \
  --key-name my-key --security-group-ids sg-12345678

# List Running Instances
aws ec2 describe-instances --filters "Name=instance-state-name,
    Values=running"
```

★ Key Features of EC2

1. **Scalable** : Launch or terminate instances quickly.
2. **Flexible** : Choose instance type, OS, storage, network.
3. **Reliable** : High availability with multiple AZs.
4. **Secure** : Security Groups, Key Pairs, IAM roles.
5. **Cost-Effective** : Pay for what you use.
6. **Customizable** : Add storage, IP, user data, monitoring.

Did You Know ?

EC2 was one of the first AWS services launched in 2006.
It's the backbone of most cloud applications !

☺

★ EC2 Components

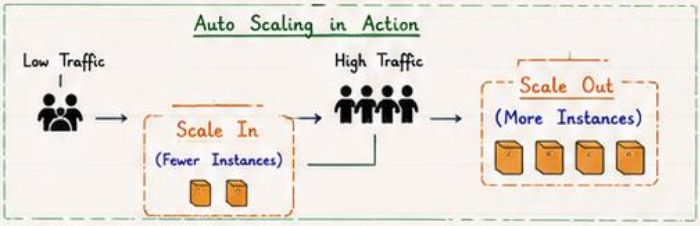
- **Instance** : A virtual server in the cloud.
- **AMI (Amazon Machine Image)** : Template used to launch instances.
- **EBS (Elastic Block Store)** : Persistent block storage.
- **Security Group** : Virtual firewall for instance.
- **Key Pair** : Used to securely connect to instance.
- **Elastic IP** : Static public IP for instances.

EC2 Instance Types (Examples)

- General Purpose : t3, t4g, m5
- Compute Optimized : c5, c6g
- Memory Optimized : r5, x1e
- Storage Optimized : i3, d2
- Burstable : t3, t2

★ What is Auto Scaling ?

Auto Scaling automatically adjusts the number of EC2 instances in your application based on demand.
It helps maintain performance and optimize cost.



★ Auto Scaling Components

- **Launch Template / Configuration** : Defines how instances are launched.
- **Auto Scaling Group (ASG)** : Group of EC2 instances that scale together.
- **Scaling Policies** : Rules that define when to scale in or scale out.
- **CloudWatch Alarms** : Monitor metrics and trigger scaling actions.
- **Health Checks** : Replace unhealthy instances automatically.

Example: Create Auto Scaling Group (AWS CLI)

```
aws autoscaling create-auto-scaling-group \
  --auto-scaling-group-name my-asg \
  --launch-template LaunchTemplateName=my-template,Version=1 \
  --min-size 2 --max-size 6 --desired-capacity 2 \
  --vpc-zone-identifier subnet-123abc,subnet-456def
```

★ Benefits of Auto Scaling

- Handles varying loads automatically.
- Ensures high availability.
- Optimizes cost by scaling in when not needed.
- Replaces unhealthy instances.
- Works across multiple Availability Zones.
- Improves application performance.

Note :

Combine EC2 + Auto Scaling + Load Balancer for highly available, fault-tolerant applications !

☺

4. S3 + Storage

★ What is S3 ?

Amazon S3 (Simple Storage Service) is an object storage service that offers industry-leading scalability, data availability, security and performance.
It is designed to store and retrieve any amount of data from anywhere on the web.

Example

```
# Create a Bucket (Using AWS CLI)
aws s3 mb s3://my-first-bucket
# List Buckets
aws s3 ls
# Upload a File
aws s3 cp file.txt s3://my-first-bucket/
```

★ Key Features of S3

1. **Durability** : 11 9's of durability for your data.
2. **Availability** : High availability and fault tolerance.
3. **Scalability** : Store unlimited amounts of data.
4. **Security** : Access control using IAM, Bucket Policies.
5. **Cost Effective** : Pay for what you use.
6. **Data Protection** : Versioning, Encryption, Lifecycle rules.
7. **Global** : Access data from anywhere in the world.

Did You Know ?

S3 was launched in 2006 and is one of the most widely used cloud storage services in the world ! ☺

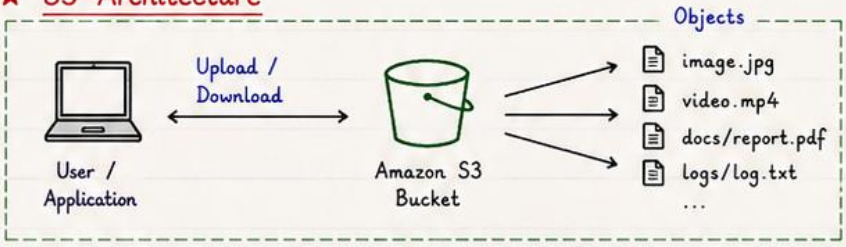
★ S3 Key Concepts

- **Bucket** : Container for storing objects.
- **Object** : Data stored in S3 (file + metadata).
- **Key** : Unique name (path) of the object in a bucket.
- **Region** : AWS Region where the bucket resides.
- **Prefix** : Folder-like concept within a bucket.
- **Versioning** : Keep multiple versions of an object.
- **Lifecycle** : Automatically move or expire objects.

S3 Storage Classes (From High to Low Cost)

- S3 Standard : Frequently accessed data
- S3 Intelligent-Tiering : Auto moves data to lower cost tiers
- S3 Standard-IA : Infrequently accessed data
- S3 One Zone-IA : Infrequent, non-critical data (single AZ)
- S3 Glacier Instant Retrieval : Archive data accessed occasionally
- S3 Glacier Flexible Retrieval : Archive data with minutes to hours retrieval
- S3 Glacier Deep Archive : Lowest cost, retrieval in hours

★ S3 Architecture



Common Use Cases

- Backup & Restore
- Data Archiving
- Static Website Hosting
- Big Data & Analytics
- Content Delivery (with CloudFront)
- Disaster Recovery

★ Best Practices

- ✓ Enable Versioning to protect against accidental deletes.
- ✓ Use Lifecycle policies to transition or expire data.
- ✓ Enable Server-Side Encryption (SSE-S3 / SSE-KMS).
- ✓ Use IAM Roles and least privilege access.
- ✓ Block public access to buckets.
- ✓ Monitor with S3 Metrics and CloudTrail.
- ✓ Use appropriate storage class to optimize cost.

Note :

S3 is NOT a file system, it is an object storage designed for 99.999999999% (11 9's) durability. ☺

5. VPC + Networking

★ What is VPC ?

Amazon VPC (Virtual Private Cloud) lets you launch AWS resources in a logically isolated virtual network that you define.

It gives you full control over your virtual networking environment, including IP addressing, routing, and security.

Example

```
# Create a VPC
aws ec2 create-vpc --cidr-block 10.0.0.0/16

# List VPCs
aws ec2 describe-vpcs
```

★ Key Components of VPC

1. **VPC** : Your virtual network in AWS (e.g., 10.0.0.0/16)
2. **Subnet** : Subdivision of VPC (Public or Private subnet).
3. **Route Table** : Controls traffic routing within the VPC.
4. **Internet Gateway** : Enables communication between VPC and the internet.
5. **NAT Gateway** : Allows instances in private subnet to access the internet.
6. **Security Group** : Virtual firewall for instances (stateful).
7. **Network ACL** : Firewall at subnet level (stateless).
8. **Elastic IP** : Static public IP for resources.
9. **VPC Peering** : Connect two VPCs privately.
10. **VPN / Direct Connect** : Connect on-premise to AWS.

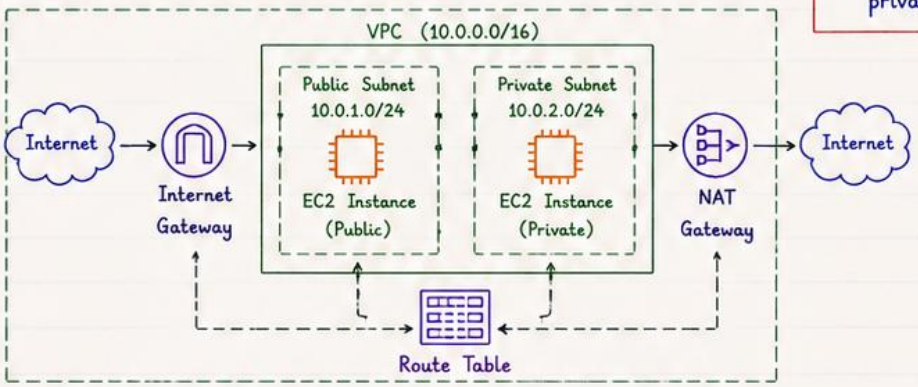
Did You Know ?

VPC was launched in 2009 and is the foundation for building secure and scalable applications on AWS. ☺

VPC Best Practices

- Use Private Subnets for backend resources.
- Use NAT Gateway instead of NAT Instance.
- Follow least privilege with Security Groups.
- Use Network ACLs for additional layer.
- Enable VPC Flow Logs for monitoring.
- Use VPC Endpoints to access AWS services privately.

★ VPC Architecture (High Level)



Common Ports

SSH (Linux)	22
RDP (Windows)	3389
HTTP	80
HTTPS	443
MySQL	3306
PostgreSQL	5432
DNS	53

★ Routing in VPC

Destination	Target	Description
10.0.0.0/16	local	Route traffic within the VPC
0.0.0.0/0	igw-xxxxxxx	Route traffic to Internet Gateway
0.0.0.0/0	nat-xxxxxxx	Route traffic to NAT Gateway
10.1.0.0/16	pcx-xxxxxxx	Route traffic to Peered VPC

★ VPC Endpoints (Types)

- Gateway Endpoint : For S3, DynamoDB (free)
- Interface Endpoint : For other AWS services (ENI creates in your subnet)
- Gateway Load Balancer Endpoint : Third-party appliances (firewalls, IDS/IPS)

Note : Design your VPC like a real-world network - Isolate, Secure, Monitor and Optimize. ☺

6. RDS + DynamoDB

★ What is RDS ?

Amazon RDS (Relational Database Service) makes it easy to set up, operate, and scale a relational database in the cloud.

It supports multiple database engines and handles backups, patching and high availability for you.

★ What is DynamoDB ?

Amazon DynamoDB is a fully managed NoSQL database service that provides fast and predictable performance with seamless scalability.

It is a key-value and document database.

Examples

```
# RDS - Create a MySQL DB instance
aws rds create-db-instance \
    --db-instance-identifier mydb \
    --engine mysql \
    --master-username admin \
    --master-user-password MyStrongPass123 \
    --allocated-storage 20 \
    --db-instance-class db.t3.micro

# DynamoDB - Create a Table
aws dynamodb create-toble \
  --table-name Users \
  --attribute-definitions AttributeName=UserId,AttributeType=S \
  --key-schema AttributeName=UserId, KeyType=HASH \
  --billing-mode PAY_PER_REQUEST
```

Did You Know ?

Aurora (an RDS engine) is up to 5x faster than standard MySQL and 1/10th the cost!

★ Key Features - RDS

1. **Managed Service** : AWS manages backups, patching, monitoring.
2. **Multiple Engines** : MySQL, PostgreSQL, MariaDB, Oracle, SQL Server, Aurora.
3. **High Availability** : Multi-AZ deployments with automatic failover.
4. **Scalability** : Scale storage and compute as needed.
5. **Security** : Encryption, IAM authentication, Security Groups.
6. **Backups** : Automated backups and point-in-time recovery.
7. **Read Replicas** : Improve read performance and availability.

★ Key Features - DynamoDB

1. **Serverless** : Fully managed, no servers to manage.
2. **Scalability** : Automatically scales to millions of requests.
3. **Performance** : Single-digit millisecond latency.
4. **Durability** : 99.999999999% (11 9's) durability.
5. **Security** : IAM, Encryption at rest and in transit.
6. **Global Tables** : Multi-region, multi-active replication.
7. **Pay-as-you-go** : Pay only for what you use.

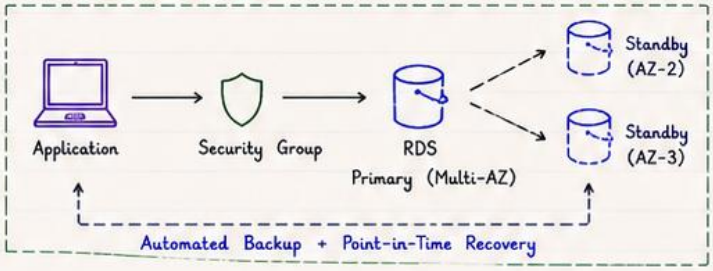
RDS vs DynamoDB

RDS (Relational)	DynamoDB (NoSQL)
Structured data	Schema-less data
SQL queries	Key-Value / Document
Joins, ACID transactions	High throughput, Low latency
Better for complex relationships	Better for large scale, real-time apps
Vertical scaling	Horizontal scaling

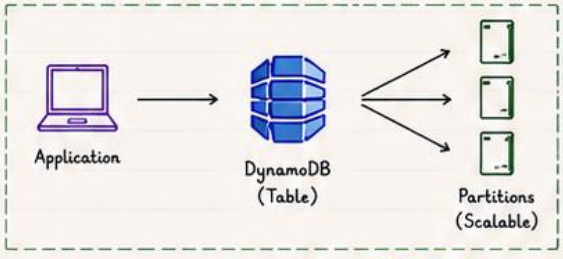
DynamoDB - Use Cases

- User profiles and sessions
- Shopping carts
- IoT device data
- Real-time analytics
- Gaming leaderboards
- Mobile and web backends

RDS Architecture



DynamoDB Architecture



Best Practices

- Use Multi-AZ for RDS production workloads.
- Regularly test backups and restore.
- Use Read Replicas for heavy read workloads.
- Enable encryption for both services.
- Use DynamoDB Auto Scaling for throughput.
- Design DynamoDB tables with proper keys.
- Monitor with CloudWatch metrics.
- Follow least privilege IAM access.

Note :

Choose RDS for complex relational data. Choose DynamoDB for big scale, high performance NoSQL workloads.

7. LAMBDA + SERVERLESS

★ What is AWS Lambda ?

AWS Lambda is a serverless compute service that lets you run code without provisioning or managing servers. You pay only for the compute time you consume.

It runs your code in response to events and automatically manages the compute resources.

Example

```
# Create a Lambda Function (Using AWS CLI)
aws lambda create-function \
  --function-name hello-world \
  --runtime python3.11 \
  --role arn:aws:iam::123456789012:role/lambda-role \
  --handler index.handler \
  --zip-file fileb://function.zip

# Invoke
aws lambda invoke --function-name hello-world output.json
```

★ Key Features of Lambda

1. **Serverless** : No servers to provision or manage.
2. **Pay-per-Use** : Pay only for the time your code runs.
3. **Auto Scaling** : Automatically scales with traffic.
4. **High Availability** : Built-in fault tolerance.
5. **Event Driven** : Runs code in response to events.
6. **Secure** : IAM integration and VPC support.
7. **Integrations** : Works with many AWS services.

Did You Know ?

Lambda was launched in 2014 and it was one of the first major serverless compute services in the world !

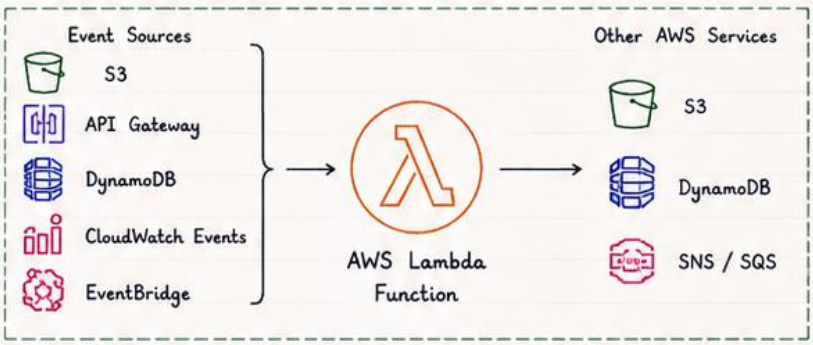
★ Lambda Components

- **Function** : Your code packaged and uploaded.
- **Handler** : Entry point of your code.
- **Runtime** : Language in which your code runs.
- **Role (IAM)** : Permissions for AWS resources.
- **Triggers** : Events that invoke your function.
- **Layers** : Reusable libraries and dependencies.
- **Environment Variables** : Configuration values for your function.

Common Use Cases

- Backends for web or mobile apps
- Real-time file processing
- Data transformation
- API backends
- Scheduled tasks (cron jobs)
- Chatbots and notifications
- IoT data processing

★ Lambda Architecture (Event Driven)



Benefits of Serverless

- Focus on code, not infrastructure.
- Faster development and deployment.
- Automatic scaling and high availability.
- Reduced operational overhead.
- Cost optimization.
- Pay only for what you use.

★ Best Practices

- ✓ Keep your functions small and focused.
- ✓ Use appropriate memory and timeout settings.
- ✓ Reuse connections and resources.
- ✓ Store configuration in Environment Variables.
- ✓ Enable CloudWatch Logs for monitoring.
- ✓ Use IAM roles with least privilege.
- ✓ Handle errors and exceptions properly.
- ✓ Test your code thoroughly.

Note :

Serverless ≠ No Servers.
It just means servers are managed by AWS, not by you !

8. CloudFront + Route 53

★ What is CloudFront ?

Amazon CloudFront is a Content Delivery Network (CDN) service that delivers your content to users with low latency and high transfer speeds.
It securely delivers data, videos, applications, and APIs through a global network of edge locations.

★ What is Route 53 ?

Amazon Route 53 is a scalable and highly available Domain Name System (DNS) web service.
It routes users to internet applications by translating domain names into IP addresses.

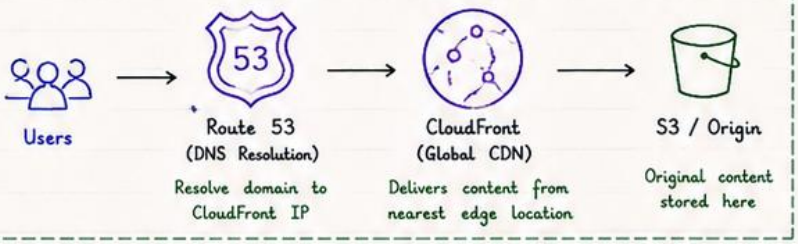
★ Key Features - CloudFront

1. Global Edge Network : Content cached at edge locations.
2. Low Latency : Delivers content closer to users.
3. Secure : HTTPS, SSL/TLS, Signed URLs & Cookies.
4. Scalable : Handles large scale traffic easily.
5. Cost Effective : Pay for what you transfer.
6. Customizable : Behaviors, caching, error pages.
7. Integration : Works with S3, EC2, ALB, API Gateway, etc.

★ Key Features - Route 53

1. Domain Registration : Register domains.
2. DNS Routing : Route traffic using various policies.
3. Health Checks : Monitor the health of your resources.
4. High Availability : Built on AWS global infrastructure.
5. Scalability : Handles billions of DNS queries.
6. Security : DNSSEC, VPC Routing (private zones).
7. Integration : Works with AWS services.

How They Work Together



Examples

```
# CloudFront - Create a Distribution
aws cloudfront create-distribution \
  --origin-domain-name my-bucket.s3.amazonaws.com \
  --default-root-object index.html \
  --price-class PriceClass_All

# Route 53 - Create a Hosted Zone
aws route53 create-hosted-zone \
  --name example.com. \
  --caller-reference $(date +%s)
```

Did You Know ?

CloudFront has 400+ edge locations in 100+ cities worldwide.
Route 53 is 100% SLA and highly available !

☺

Route 53 Routing Policies

- Simple Routing : Route traffic to a single resource.
- Weighted Routing : Distribute traffic based on weights.
- Latency Routing : Route based on user latency.
- Failover Routing : Automatic failover to standby.
- Geolocation Routing : Route based on user location.
- Geoproximity Routing : Route based on resource location.
- Multivalue Answer : Return multiple IPs in response.

Common Use Cases

- Static Website Hosting
- Video Streaming
- Software / Download Distribution
- API Acceleration
- Secure Content Delivery
- Global Applications

Best Practices

- ✓ Use CloudFront for all public content.
- ✓ Enable HTTPS and Redirect HTTP to HTTPS.
- ✓ Use Origin Access Control (OAC) for S3 security.
- ✓ Configure proper Caching Policies.
- ✓ Use Route 53 Health Checks with Failover.
- ✓ Choose right Routing Policy for your use case.
- ✓ Enable DNSSEC for added security.
- ✓ Monitor with CloudWatch & Route 53 Metrics.

Note :

Route 53 finds the right destination, CloudFront delivers it the fastest and most secure way !

☺

9. Load Balancer + API Gateway

★ What is Load Balancer ?

Elastic Load Balancing (ELB) automatically distributes incoming traffic across multiple targets such as EC2 instances, containers, and IP addresses.
It improves application availability, scalability and fault tolerance.

★ What is API Gateway ?

Amazon API Gateway is a fully managed service that makes it easy to create, publish, maintain, monitor, and secure APIs at any scale.
It acts as a front door for applications to access backend services.

★ Key Features - Load Balancer

1. **High Availability** : Spans multiple AZs automatically.
2. **Scalability** : Handles millions of requests.
3. **Security** : Integrates with Security Groups, WAF.
4. **Health Checks** : Monitors target health.
5. **Multiple Types** : ALB (Layer 7), NLB (Layer 4), Gateway LB.
6. **Routing Rules** : Path-based, Host-based, Query-based.
7. **SSL/TLS Termination** : Offloads SSL to LB.
8. **Sticky Sessions** : Maintain user session affinity.

★ Key Features - API Gateway

1. **API Management** : Create and manage APIs easily.
2. **Security** : IAM auth, Cognito, API Keys, WAF.
3. **Throttling** : Control request rate.
4. **Caching** : Reduce backend load.
5. **Monitoring** : CloudWatch metrics and logs.
6. **Integration** : Lambda, HTTP, AWS services, VPC Link.
7. **Versioning & Stages** : Manage multiple versions.
8. **Developer Experience** : SDK generation, Docs, CORS.

Examples

```
# Application Load Balancer - Create (AWS CLI)
aws elbv2 create-load-balancer \
    --name my-alb \
    --subnets subnet-1 subnet-2 \
    --security-groups sg-12345678 \
    --type application

# API Gateway - Create a REST API
aws apigateway create-rest-api \
    --name MyAPI \
    --description "My first API"
```

Did You Know ?

ALB operates at Layer 7 (Application) and can route based on content, path, host headers and more !
API Gateway can handle millions of requests per second.

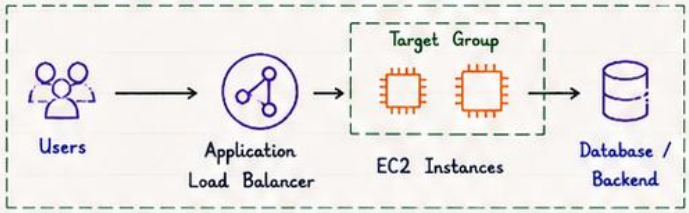
ALB vs NLB

Feature	ALB (Layer 7)	NLB (Layer 4)
OSI Layer	7 (Application)	4 (Transport)
Protocol	HTTP, HTTPS, gRPC	TCP, UDP, TLS, TCP
Use Case	Web apps, APIs	High perf, TCP apps
Routing	Content-based	IP/Port-based
Target Type	Instance, IP, Lambda	Instance, IP
Performance	Good	Extreme

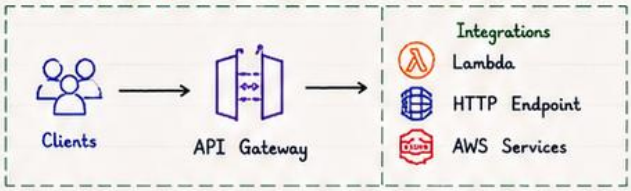
API Gateway - Use Cases

- Mobile / Web backend
- Microservices API facade
- Internet of Things (IoT)
- Partner / Third-party access
- Serverless applications
- B2B integrations

Load Balancer Architecture (ALB)



API Gateway Architecture



Best Practices

- ✓ Use ALB for HTTP/HTTPS based applications.
- ✓ Use NLB for extreme performance TCP workloads.
- ✓ Enable Cross-Zone Load Balancing.
- ✓ Configure proper Health Checks.
- ✓ Use API Gateway with IAM + API Keys.
- ✓ Enable Caching to reduce backend calls.
- ✓ Set Throttling limits to protect backend.
- ✓ Monitor with CloudWatch & enable access logs.

Note :

Load Balancer distributes traffic.
API Gateway manages, secures and routes API requests.

10. CloudWatch + CloudTrail + Monitoring

★ What is Monitoring in AWS ?

Monitoring helps you collect, track and analyze metrics, logs and events from your AWS resources and applications. It improves performance, ensures security and helps in troubleshooting.

★ Key Services Overview

1. **CloudWatch** : Monitors resources, metrics, logs and sets alarms.
2. **CloudTrail** : Tracks API calls and user activity for governance and auditing.
3. **Monitoring** : The overall practice of collecting, visualizing and alerting on data.

Examples (AWS CLI)

```
# CloudWatch - Create Alarm
aws cloudwatch put-metric-alarm \
  --alarm-name CPU-High \
  --metric-name CPUUtilization \
  --namespace AWS/EC2 \
  --statistic Average --period 300 \
  --threshold 80 --comparison-operator GreaterThanThreshold \
  --evaluation-periods 2 --alarm-actions arn:aws:sns:us-east-1:123456789012:NotifyMe

# CloudTrail - Lookup Events
aws cloudtrail lookup-events \
  --lookup-attributes AttributeKey=Username,AttributeValue=nikhil
```

Did You Know ?

CloudTrail records events for free for 90 days ! After that you can store them in S3 for long-term retention.

★ CloudWatch - Key Features

1. **Metrics** : Collects metrics from AWS services and applications.
2. **Alarms** : Sends notifications when metrics cross thresholds.
3. **Dashboards** : Visualize metrics in real-time.
4. **Logs** : Centralized log management (CloudWatch Logs).
5. **Events** : Rule-based events for automation.
6. **Insights** : Analyze logs using CloudWatch Logs Insights.
7. **Container Insights** : Monitor containers, ECS, EKS performance.

Common CloudWatch Metrics

- CPUUtilization (EC2)
- StatusCheckFailed (EC2)
- RequestCount (ELB)
- Latency (API Gateway)
- 5XXError (API Gateway)
- Invocations (Lambda)
- Throttles (Lambda)

★ CloudTrail - Key Features

1. Records API calls as events.
2. Helps in Security analysis and Compliance.
3. Stores events in S3 (can be integrated with CloudWatch Logs).
4. Supports Multi-region and Multi-account trails.
5. Detects Unauthorized or suspicious activities.

Example CloudTrail Event (JSON)

```
{
  "eventTime": "2024-05-20T10:15:30Z",
  "eventSource": "ec2.amazonaws.com",
  "eventName": "RunInstances",
  "userIdentity": { "type": "IAMUser", "userName": "nikhil" },
  "sourceIPdddress": "103.21.45.67"
}
```

★ Monitoring Best Practices

- ✓ Set meaningful Alarms and avoid Alarm fatigue.
- ✓ Use Dashboards for Quick visibility.
- ✓ Keep Logs centralized and searchable.
- ✓ Enable CloudTrail in all regions.
- ✓ Use Tags to filter and organize metrics and logs.
- ✓ Review and delete old logs to control costs.

How They Work Together



★ Architecture Overview



Use Cases

- Detect unusual API activity
- Monitor application performance
- Trigger auto recovery actions
- Audit and compliance reporting
- Troubleshoot issues quickly

Note : Monitoring is not just about collecting data, but using it to take action and improve your systems.



11. SQS + SNS + Event-Driven Architecture

What are SQS and SNS ?

Amazon SQS (Simple Queue Service)
A fully managed message queue service that decouples and scales microservices, distributed systems and serverless applications.

Amazon SNS (Simple Notification Service)
A fully managed pub/sub messaging service for sending messages to multiple subscribers (email, SMS, HTTP, SQS, Lambda, etc.).

Why Use Them Together ?

SNS is great for broadcasting (one-to-many).
SQS is great for decoupling and processing messages reliably (point-to-point).
Together they enable scalable, reliable, event-driven architectures.

Examples (AWS CLI)

```
# 1. Create an SQS Queue
aws sqs create-queue \
    --queue-name order-processing-queue \
    --attributes VisibilityTimeout=30,MessageRetentionPeriod=345600

# 2. Create an SNS Topic
aws sns create-topic \
    --name order-events

# 3. Subscribe SQS Queue to SNS Topic
aws sns subscribe \
    --topic-arn arn:aws:sns:us-east-1:123456789012:order-events \
    --protocol sqs \
    --notification-endpoint arn:aws:sqs:us-east-1:123456789012:order-processing-queue

# 4. Publish a Message to SNS Topic
aws sns publish \
    --topic-arn arn:aws:sns:us-east-1:123456789012:order-events \
    --message "New order received!"
```

Did You Know ?

SNS can fan-out a message to thousands of subscribers in milliseconds !

Key Features

SQS

- Decouples producers and consumers
- Standard & FIFO queues
- At-least-once delivery
- Long polling for efficiency
- Message visibility timeout
- Dead Letter Queue (DLQ)
- Encryption, IAM, VPC endpoints

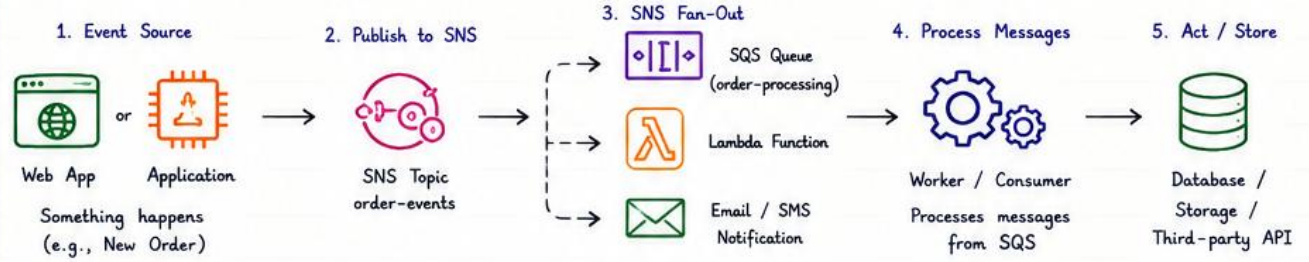
SNS

- Pub/Sub (one-to-many)
- Multiple protocols (HTTP, Email, SMS, Lambda, SQS, etc.)
- Message filtering
- Delivery retries & DLQ
- Cross-account & cross-region
- Serverless & highly scalable

SQS vs SNS

Feature	SQS	SNS
Model	Queue (Point-to-Point)	Pub/Sub (One-to-Many)
Purpose	Decouple & buffer messages	Broadcast messages to multiple subscribers
Delivery	At-least-once	Best-effort
Pull / Push	Pull (Consumers poll)	Push
Persistence	Messages stored in queue	No persistence (fire and forget)
Use Cases	Work queues, background jobs, decoupling	Notifications, fan-out, event distribution

How They Work Together (Event-Driven Flow)



Decoupled • Scalable • Reliable • Event-Driven

Common Use Cases

- ☑ Order processing system
- ☑ Email / SMS notifications
- ☑ Image/video processing pipeline
- ☑ IoT data ingestion & alerts
- ☑ Microservices communication
- ☑ Audit logging & compliance events

Build systems that react to events, not just requests !

Best Practices

- ☑ Use FIFO queues when order matters.
- ☑ Use Message Attributes & Filtering.
- ☑ Enable DLQ for failed messages.
- ☑ Set appropriate Visibility Timeout.
- ☑ Use Long Polling to reduce empty responses.
- ☑ Monitor with CloudWatch metrics & alarms.
- ☑ Follow least privilege IAM policies.

SQS Message Lifecycle

Message Sent → Stored in Queue → Consumer Receives → Processed Successfully → Message Deleted

Pro Tip

Use SNS to broadcast events and SQS to process them reliably.
This combination is the heart of serverless, event-driven systems ! 💪

Remember

SNS = Tell Many
SQS = Work Queue
Together = Powerful Architecture

12. Top 100 AWS Interview Questions + Quick Cheat Sheet

☆ These questions are commonly asked in AWS Interviews.

1. What is AWS?
2. What is the AWS Global Infrastructure?
3. What is a Region, AZ and Edge Location?
4. What is IAM?
5. What are IAM policies and types?
6. What is the difference between User, Group and Role?
7. What is MFA in AWS?
8. What is the root user?
9. What is EC2?
10. What are EC2 instance types?
11. What is Amazon Machine Image (AMI)?
12. What is the difference between Stop vs Terminate?
13. What is EBS?
14. What are EBS volume types?
15. What is the difference between EBS and Instance Store?
16. What is Elastic IP?
17. What is Placement Group?
18. What is Auto Scaling?
19. What is the difference between Scale Out vs Scale Up?
20. What is ELB?
21. What are the types of Load Balancers?
22. What is the difference between ALB, NLB and CLB?
23. What is Health Check in ELB?
24. What is S3?
25. What are S3 storage classes?
26. What is the difference between S3 Standard vs IA?
27. What is Versioning in S3?
28. What is Lifecycle Policy in S3?
29. What is Cross-Region Replication (CRR)?
30. What is CloudFront?
31. What is the difference between CloudFront and S3?
32. What is Route 53?
33. What is DNS in Route 53?
34. What are Routing Policies in Route 53?
35. What is VPC?
36. What are Subnet types?
37. What is the difference between Public vs Private Subnet?
38. What is Internet Gateway?
39. What is NAT Gateway?
40. What are Security Groups?
41. What are NACLs?
42. What is VPC Peering?
43. What is RDS?
44. What are RDS DB engines?
45. What is Read Replica in RDS?
46. What is Multi-AZ in RDS?
47. What is DynamoDB?
48. What are DynamoDB read/write capacities?
49. What is DynamoDB Global Tables?
50. What is Aurora?
51. What is AWS Lambda?
52. What are the benefits of Lambda?
53. What is the difference between Lambda and EC2?
54. What is a Cold Start in Lambda?
55. What is API Gateway?
56. What are API Gateway types?
57. What is the difference between REST and HTTP API?
58. What are Authorizers in API Gateway?
59. What is SQS?
60. What are SQS queue types?
61. What is the difference between Standard vs FIFO queues?
62. What is SNS?
63. What are SNS delivery protocols?
64. What is the difference between SQS and SNS?
65. What is CloudWatch?
66. What are CloudWatch Metrics?
67. What is CloudWatch Logs?
68. What are CloudWatch Alarms?
69. What is CloudTrail?
70. What does CloudTrail log?
71. What is AWS Config?
72. What is the difference between CloudTrail and Config?
73. What is Auto Scaling Group?
74. What are Auto Scaling Policies?
75. What is Elastic Beanstalk?
76. What is ECS?
77. What is EKS?
78. What is the difference between ECS and EKS?
79. What is Fargate?
80. What is IAM Role vs Instance Profile?
81. What is KMS?
82. What is the difference between SSE-S3, SSE-KMS, SSE-C?
83. What is VPC Endpoint?
84. What are the types of VPC Endpoints?
85. What are CloudFormation?
86. What are CloudFormation Templates?
87. What is Terraform and how is it different?
88. What is the AWS Well-Architected Framework?
89. What are the 6 pillars of Well-Architected?
90. How do you make an application Highly Available?
91. How do you design for Scalability?
92. How do you ensure Security in AWS?
93. How do you reduce costs in AWS?
94. How do you monitor an application in AWS?
95. How do you secure data in transit?
96. How do you secure data at rest?
97. What is the AWS Shared Responsibility Model?
98. How do you implement Disaster Recovery?
99. How do you replicate data across regions?
100. How do you troubleshoot performance issues?

Quick Cheat Sheet

Compute
- EC2 – Virtual Servers
- Lambda – Serverless Functions
- ECS / EKS – Containers
- Auto Scaling – Scale Instances

Storage
- S3 – Object Storage
- EBS – Block Storage
- EFS – File Storage
- Glacier – Archival Storage

Database
- RDS – Managed Relational DB
- DynamoDB – NoSQL DB
- Aurora – AWS Relational DB
- ElastiCache – In-Memory Cache

Networking & CDN
- VPC – Private Network
- Route 53 – DNS Service
- CloudFront – CDN
- ELB – Load Balancing

Integration
- SQS – Message Queue
- SNS – Notifications
- EventBridge – Event Bus
- API Gateway – APIs

Monitoring & Management
- CloudWatch – Monitoring
- CloudTrail – API Logs
- AWS Config – Resource Tracking
- CloudFormation – IaC

Security
- IAM – Access Management
- KMS – Key Management
- Security Groups – Firewall
- WAF – Web Application Firewall

AWS Global Infrastructure
- Regions (36+)
- Availability Zones (Multiple per Region)
- Edge Locations (For CloudFront)

Interview Tips

- ✓ Understand the core concepts deeply.
- ✓ Know the use cases of each service.
- ✓ Explain with real-world examples.
- ✓ Think in terms of Scalability, Availability, Security and Cost.
- ✓ Draw architectures when possible.

Well-Architected Pillars

- Operational Excellence
- Security
- Reliability
- Performance Efficiency
- Cost Optimization
- Sustainability

Memory Trick (To Remember Key Services)

"CES + N + D + M + S"

C	E	N	D	Fi	M	S
Compute	Storage	Networking	Database	Messaging	Messaging	Security & Monitoring
EC2	S3	VPC	RDS	SQS	SQS	IAM
Lambda	EBS	Route 53	DynamoDB	SNS	SNS	CloudWatch
ECS / EKS	EFS	CloudFront	Aurora	EventBridge	CloudWatch	CloudTrail

Remember:

★ AWS is huge, but interviews focus on core services, use cases, and how you design solutions.

You've got this! Keep Building!